SESSION 12 OF 25 · LIVE API CALLS

Calling Real APIs.

Your apps already store data and read text. Today they reach out — live weather, currency rates, news, anything on the internet. The day your apps stop being self-contained.

PROGRAM
Python Internship

LED BY
Zlaark

SESSION
12 of 25

AGENDA · DAY 12

How we'll spend our 90 minutes today.

APIs, the requests library, JSON responses, error handling, Streamlit integration, and a live weather dashboard. The bridge to real-world software.

00:00–00:10

Day 11 Recap & Q&A

Regex, PDF extraction, Word docs, the resume contact extractor. Quick check.

00:10–00:25

What is an API? + requests.get

The restaurant analogy. GET vs POST. The 4-line requests pattern. JSON responses.

00:25–00:40

Query Parameters + Error Handling

URL params, status codes, try/except, timeouts. The real-world pattern.

00:40–00:50

APIs in Streamlit + Free APIs

st.cache_data for API calls. A list of free APIs you can use today — no key needed.

00:50–01:00

Reading API Documentation

How to read an API doc, find the endpoint, figure out params, understand the response.

01:00–01:30

Exercise: Live Weather Dashboard

Call Open-Meteo API (free, no key), show temperature + wind + humidity for any city. Combines requests + JSON + Streamlit.

Yesterday your apps learned to read. Today they learn to reach out.

Quick check that Day 11's resume extractor runs. Today's exercise uses the same requests + JSON pattern — but pulls live data from the internet.

WHAT WE COVERED YESTERDAY

- ✓ **Regex basics** — re.findall, re.search, re.sub
- ✓ **Regex syntax** — anchors, quantifiers, character classes, groups
- ✓ **Escaping & greedy vs lazy** — the two regex gotchas
- ✓ **PDF extraction with PyPDF2** — pull text out of PDFs
- ✓ **Word doc extraction with python-docx** — paragraphs, tables, styles

CHECK YOUR LAPTOP NOW

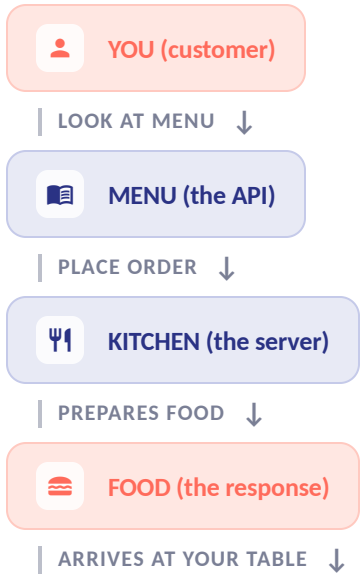
```
# You should have Day 11's resume extractor:
$ ls day11/
resume_extractor.py
sample_resume.pdf

# Run it — should extract contacts:
$ python resume_extractor.py
# → Email: aarav.sharma@example.com
# → Phone: +91 98765 43210
# → Name: Aarav Sharma
```

IF THE EXTRACTOR DOESN'T RUN — raise your hand. Today builds on regex + file reading, so you need yesterday's patterns working.

An API is a waiter. You order, they bring food.

API stands for Application Programming Interface. It's a set of rules for how two programs talk to each other. You send a request, the API sends back data. Like ordering food at a restaurant.

THE RESTAURANT



You don't go into the kitchen. You don't cook. You read the menu, order, and food arrives. The API is the menu + the waiter.

THE TECHNICAL VERSION

YOU → Your Python script
You write code that sends an HTTP request.

MENU → The API documentation
Tells you what you can ask for and how.

KITCHEN → The API server
A computer somewhere on the internet that processes your request.

FOOD → The JSON response
Data comes back as JSON (which you learned on Day 9).

KEY IDEA An API is a contract. You send a request in the right format, the API sends back data in a predictable format. No negotiation — you follow the rules, you get data.

GET — read data. POST — send data. That's 95% of API calls.

HTTP has many methods, but you'll use two. GET fetches data (like opening a URL). POST sends data (like submitting a form). Today is all GET.

GET

Read data from the server

YOU SEND	a URL (and optional params)
SERVER RETURNS	data (usually JSON)
LIKE	opening a webpage, searching Google
EXAMPLE	`requests.get("https://api.weather.com/v1/current")`
SAFE TO REPEAT	calling GET 100 times gives the same result
TODAY	**100% GET requests**

POST

Send data to the server

YOU SEND	a URL + a body (data to create/modify)
SERVER RETURNS	confirmation (usually JSON)
LIKE	submitting a form, posting on social media
EXAMPLE	`requests.post("https://api.example.com/users", json={"name": "Aarav"})`
NOT SAFE	calling POST twice creates two records
USED ON	**Day 13+: sending resume text to an AI API**

TODAY — We only use GET. You send a URL, the API sends back JSON. POST comes later (Day 13 AI exercises, when you send text to be analyzed).

HANDS-ON · THE 4-LINE PATTERN

Four lines. That's an API call.

The requests library makes HTTP calls simple. Install it, import it, call get(), parse the JSON. Four lines — and your app has live data from the internet.

```python
# first_api_call.py
# Install first: pip install requests
import requests

# 1. Call the API (GET request)
url = "https://api.open-meteo.com/v1/forecast"
response = requests.get(url, params={
    "latitude": 31.63,   # Amritsar
    "longitude": 74.87,
    "current": "temperature_2m,wind_speed_10m"
})

# 2. Check it worked
print(response.status_code)  # 200 = OK

# 3. Parse the JSON response
data = response.json()

# 4. Use the data
temp = data["current"]["temperature_2m"]
```

1. **`requests.get(url, params={...})`** — Sends a GET request. params become ?key=value in the URL. Returns a Response object.
2. **`response.status_code`** — 200 = OK. 404 = not found. 500 = server error. Always check before using the data.
3. **`response.json()`** — Parses the JSON body into a Python dict. Same as json.loads() from Day 9 — but built into requests.
4. **`data["current"]["temperature_2m"]`** — Navigate the nested dict. The response structure depends on the API — check the docs.

THE RESPONSE OBJECT

- **`response.status_code`** → `200`
- **`response.text`** → raw JSON string
- **`response.json()`** → Python dict
- **`response.headers`** → metadata
- **`response.url`** → final URL
- **`response.elapsed`** → time taken

Cross-ref: Day 9 — `response.json()` is the same as `json.loads(response.text)`. The requests library just wraps it for you.

OPEN-METEO — This API is free, needs no key, and works right now. We use it in today's exercise.

CONCEPT · QUERY PARAMETERS

Query params — how you customize the API call.

The ? in a URL starts the query string. Each param is key=value, joined by &. The requests library handles this for you — pass a dict, it builds the URL.

URL ANATOMY

```
https://api.open-meteo.com/v1/forecast?latitude=31.63&longitude=74.87&current=temperature_2m
|                                     |             |            |       |
|                                     |             |            |       └─ param 3
|                                     |             |            └─ separator (&)
|                                     |             └─ param 2
|                                     └─ ? starts query string
└─ base URL (the endpoint)
```

Each param customizes the request. The API docs tell you which params are available.

```python
# params_demo.py
import requests

# ✗ BAD — manually building the URL string
url = "https://api.open-meteo.com/v1/forecast?lat=31.63&lon=74.87&curren
response = requests.get(url)
# Hard to read. Easy to make typos. Hard to change params dynamically.

# ✓ GOOD — pass params as a dict
url = "https://api.open-meteo.com/v1/forecast"
response = requests.get(url, params={
    "latitude": 31.63,
    "longitude": 74.87,
    "current": "temperature_2m,wind_speed_10m,relative_humidity_2m"
})
# requests builds the URL for you:
# → ...?latitude=31.63&longitude=74.87&current=temperature_2m,wind_speed

# You can also build params dynamically:
city_coords = {"Amritsar": (31.63, 74.87), "Delhi": (28.61, 77.21)}
city = "Amritsar"
lat, lon = city_coords[city]
response = requests.get(url, params={"latitude": lat, "longitude": lon,
```

PATTERN — Pass params as a dict to `requests.get(url, params={...})`. The library handles URL encoding, special characters, and joining with &. Never build query strings manually.

PATTERN · ERROR HANDLING

APIs fail. Your code shouldn't crash when they do.

The internet is unreliable. Servers go down. URLs change. Your internet drops. Every API call needs error handling — or your app crashes in front of the user.

```python
# safe_api_call.py
import requests

try:
    response = requests.get(
        "https://api.open-meteo.com/v1/forecast",
        params={"latitude": 31.63, "longitude": 74.87, "current": "temperature_2m"},
        timeout=10  # ← always set a timeout!
    )
    response.raise_for_status()  # raises on 4xx/5xx
    data = response.json()
    print(f"Temperature: {data['current']['temperature_2m']}°C")

except requests.exceptions.Timeout:
    print("Request timed out. Check your internet.")
except requests.exceptions.ConnectionError:
    print("Could not connect. Is the internet working?")
except requests.exceptions.HTTPError as e:
    print(f"API returned an error: {e.response.status_code}")
except Exception as e:
    print(f"Something went wrong: {e}")
```

1. **`timeout=10`** — Always set a timeout. Without it, **`requests.get()`** can hang FOREVER if the server doesn't respond.
2. **`response.raise_for_status()`** — Raises an HTTPError if status_code is 4xx or 5xx. Catches bad URLs, auth failures, server crashes.
3. **`Multiple except blocks`** — Handle different failure modes differently. Timeout ≠ connection error ≠ HTTP error.

STATUS CODES

CODE	MEANING	WHAT TO DO
200	**OK**	Use the data
400	**Bad Request**	Check your params
401	**Unauthorized**	API key missing/invalid
403	**Forbidden**	You don't have access
404	**Not Found**	Wrong URL/endpoint
429	**Too Many Requests**	Rate limited — slow down
500	**Server Error**	API is broken — try later

THE 3 RULES — (1) Always set `timeout=`. (2) Always check `status_code` or use `raise_for_status()`. (3) Always wrap in `try/except`. Skip any of these and your app will crash at the worst time.

PATTERN · APIS IN STREAMLIT

Streamlit + APIs — the run-loop meets the internet.

Every widget interaction re-runs your script (Day 4's run-loop). If you call an API on every rerun, you hit rate limits fast. The fix: `st.cache_data`.

```python
# weather_app.py
import streamlit as st
import requests

st.title("Live Weather")

# Cache the API call — don't re-fetch on every rerun
@st.cache_data(ttl=600)  # cache for 10 minutes
def get_weather(lat: float, lon: float) -> dict:
    response = requests.get(
        "https://api.open-meteo.com/v1/forecast",
        params={"latitude": lat, "longitude": lon, "current": "temperature_2m"},
        timeout=10
    )
    response.raise_for_status()
    return response.json()

# User picks a city
city = st.selectbox("City", ["Amritsar", "Delhi", "Mumbai"], key="city")
coords = {"Amritsar": (31.63, 74.87), "Delhi": (28.61, 77.21), "Mumbai": (19.08, 72.88)}
lat, lon = coords[city]

# Call the cached function
data = get_weather(lat, lon)
temp = data["current"]["temperature_2m"]
st.metric("Temperature", f"{temp}°C")
```

CACHING + THE GOTCHA

st.cache_data

Caches the return value. Same arguments = same result, no API call. `ttl=600` = cache expires after 10 min.

The gotcha

Without caching, EVERY widget interaction calls the API. Type a letter → API call. Click a button → API call. You'll hit rate limits in seconds.

ttl parameter

`ttl=600` means cache for 600 seconds (10 min). After that, the next call re-fetches. Set ttl based on how fresh the data needs to be.

Function must be pure

The cached function must return the same output for the same input. No side effects, no random values, no reading files that change.

THE PATTERN — Wrap every API call in `@st.cache_data(ttl=N)`. The function takes params, returns data, and Streamlit caches it. Same params = cached result, no API call. Different params = fresh API call.

REFERENCE · FREE APIS

Five free APIs. No key. No payment. Use them today.

Most APIs require an API key (registration + token). These five are completely free and need no key. Perfect for learning, prototyping, and the group project.

Open-Meteo

PROVIDES Weather data — temperature, wind, humidity, precipitation for any lat/lon.

`https://api.open-meteo.com/v1/forecast`

KEY NEEDED
NO — completely free, no registration

Used in: Today's exercise (weather dashboard)

ExchangeRate-API

PROVIDES Live currency exchange rates — USD to INR, EUR to GBP, etc.

`https://open.er-api.com/v6/latest/USD`

KEY NEEDED
NO — free tier, no registration

Used in: Possible group project (expense tracker with currency)

JSONPlaceholder

PROVIDES Fake data for testing — posts, comments, users, todos. 100 items each.

`https://jsonplaceholder.typicode.com/posts`

KEY NEEDED
NO — designed for testing and learning

Used in: Learning — practice API calls without real data

GitHub API

PROVIDES GitHub repo data — user info, repositories, commits, issues.

`https://api.github.com/users/USERNAME/repos`

KEY NEEDED
NO for basic use (rate limited to 60/hr). **YES** for higher limits.

Used in: Showing your own GitHub repos in a portfolio app

Quotable

PROVIDES Random quotes — by author, by tag, random.

`https://api.quotable.io/random`

KEY NEEDED
NO — completely free

Used in: A quote-of-the-day app — simple and satisfying

SKILL · READING API DOCS

Every API is different. The docs tell you how.

You won't memorize API endpoints. You'll read the docs, find 4 things, and write the call. Here's the pattern for decoding any API.

THE 4 THINGS TO FIND

1 — The base URL (endpoint)

Where do I send the request? Usually in the first section of the docs. Example: `https://api.open-meteo.com/v1/forecast`

2 — The required parameters

What must I pass for the API to work? Usually latitude/longitude, or a query, or an ID. The docs list required vs optional params.

3 — Authentication

Do I need a key? If yes, how do I pass it — as a query param (`?api_key=...`), in a header (`Authorization: Bearer ...`), or in a cookie?

4 — The response structure

What does the JSON look like? The docs show a sample response. You navigate it like a dict: `data["current"]["temperature_2m"]`.

WORKED EXAMPLE: OPEN-METEO

STEP 1 — BASE URL

`https://api.open-meteo.com/v1/forecast`

STEP 2 — REQUIRED PARAMS

latitude *(float)* — **required**
longitude *(float)* — **required**
current *(string)* — which weather variables

STEP 3 — AUTHENTICATION

None needed. Completely free.

STEP 4 — RESPONSE STRUCTURE

```
{
  "current": {
    "temperature_2m": 28.5,
    "wind_speed_10m": 12.3,
    "relative_humidity_2m": 65
  }
}
```

```
# Access: data["current"]["temperature_2m"]
```

THE PATTERN — Find the URL, find the params, check auth, study the response. Then write `requests.get(url, params={...})`, check `status_code`, parse `.json()`. Four steps, any API.

03 / 03

Build a live weather dashboard.

Call the Open-Meteo API (free, no key), show temperature + wind + humidity for any city.
Combines requests + JSON + Streamlit + caching. The real-world app pattern.

BRIEF (5 MIN) BUILD (20 MIN) EXTEND (5 MIN)

EXERCISE · BRIEF · 5 MIN

Call Open-Meteo. Show live weather. For any city.

Build a Streamlit app that calls the Open-Meteo API (free, no key) and shows current temperature, wind speed, and humidity for a city the user picks. Real data, live from the internet.

TASK THE BRIEF

Build a Streamlit app called `weather_dashboard.py` that calls the Open-Meteo API and shows current weather for Amritsar, Delhi, or Mumbai.

- ✓ Use `requests.get()` to call `https://api.open-meteo.com/v1/forecast`
- ✓ Pass params: `latitude, longitude, current=temperature_2m,wind_speed_10m,relative_humidity_2m`
- ✓ Wrap the API call in a function with `@st.cache_data(ttl=600)`
- ✓ Handle errors with `try/except` (timeout, connection, HTTP error)
- ✓ Show 3 `st.metric` cards: Temperature, Wind Speed, Humidity
- ✓ Let the user pick a city with `st.selectbox`

BY THE END Every student has a working weather dashboard that pulls live data from the internet. The real-world app pattern.

SCAFFOLD WEATHER_DASHBOARD.PY

```python
# weather_dashboard.py — starting scaffold
import streamlit as st
import requests

st.title("Live Weather Dashboard")

# City coordinates
CITIES = {
    "Amritsar": (31.63, 74.87),
    "Delhi": (28.61, 77.21),
    "Mumbai": (19.08, 72.88),
}

# TODO: @st.cache_data(ttl=600)
def get_weather(lat: float, lon: float) -> dict:
    # TODO: call the API with requests.get
    # TODO: check response.raise_for_status()
    # TODO: return response.json()
    pass

# User picks a city
city = st.selectbox("Choose a city", list(CITIES.keys()), key="city")
lat, lon = CITIES[city]

# TODO: call get_weather(lat, lon)
# TODO: extract temp, wind, humidity from the response
# TODO: st.metric("Temperature", f"{temp}°C")
# TODO: st.metric("Wind Speed", f"{wind} km/h")
# TODO: st.metric("Humidity", f"{humidity}%")
```

EXERCISE · SOLUTION WALKTHROUGH

One way to solve it. There are others — this is just clean.

Walk through this line by line. Then go back to your own version and compare.

SOLUTION WEATHER_DASHBOARD.PY

```python
import streamlit as st
import requests

① st.title("Live Weather Dashboard")

② CITIES = {
    "Amritsar": (31.63, 74.87),
    "Delhi": (28.61, 77.21),
    "Mumbai": (19.08, 72.88),
}

③ @st.cache_data(ttl=600)
def get_weather(lat: float, lon: float) -> dict:
    try:
        response = requests.get(
            "https://api.open-meteo.com/v1/forecast",
            params={"latitude": lat, "longitude": lon,
                    "current": "temperature_2m,wind_speed_10m,relative_humidity_2m"},
            timeout=10
        )
        response.raise_for_status()
        return response.json()
    except requests.exceptions.RequestException as e:
        st.error(f"API error: {e}")
        return {}

④ city = st.selectbox("City", list(CITIES.keys()), key="city")
lat, lon = CITIES[city]
```

LINE BY LINE

1 Title

Standard `st.title`. The page heading.

2 City coordinates

A dict mapping city names to `(lat, lon)` tuples. The user picks a city, you look up coords.

3 Cached API function

`@st.cache_data(ttl=600)` caches for 10 min. try/except catches all request errors. Returns `{}` on failure.

4 User picks a city

`st.selectbox` with keyed widget. Looks up lat/lon from the dict. Day 5 pattern.

5 Display metrics

3 columns, each with `st.metric`. Checks data exists before accessing (the `if data` check). Error handling shows `st.warning` if API failed.

THE PIPELINE User picks city → look up coords → call cached API → parse JSON → show metrics. This is the pattern behind every data-fetching app: input → API call → display.

EXERCISE · EXTEND + WHAT'S NEXT

Finished early? Add a forecast. Or peek at tomorrow.

Three extension ideas for fast students. Plus a preview of Days 13–14 — A First Taste of AI.

EXTEND EXTENSION IDEAS

1 Add a 7-day forecast

Add the `"daily"` param to the API call: `params={..., "daily": "temperature_2m_max,temperature_2m_min"}`. Show a 7-day forecast table or line chart (Day 7).

2 Add more cities

Add 5+ more cities to the `CITIES` dict. Or use a free geocoding API to look up coords by city name: `requests.get("https://geocoding-api.open-meteo.com/v1/search", params={"name": city_name})`.

3 Add a temperature chart

Fetch hourly data for the next 24 hours, show as a `st.line_chart`. `params={..., "hourly": "temperature_2m"}`. Combines Day 7 charts with today's API call.

A FIRST TASTE OF AI WHAT'S NEXT · DAYS 13–14

Day 13 — TF-IDF + text similarity

How computers compare two pieces of text. The math behind resume matchers and plagiarism checkers.

Day 14 — Resume matcher

Build a resume-vs-job-description matcher. Uses Day 11's PDF extraction + Day 13's TF-IDF. A real AI-powered tool.

The group project connection

4 of 10 project ideas use these AI techniques. Today's API skills connect to calling AI APIs (like OpenAI) in the group project.

TOMORROW Your apps start feeling smart. TF-IDF, similarity matching, and the same technique behind real AI tools. The foundation for the group project.

IF YOU'RE STUCK Check: (1) internet is on, (2) `requests` is installed (`pip install requests`), (3) the API URL is correct, (4) you're passing `params` not building the URL manually.

Q&A · CLOSING · 5 MIN

Your apps now reach the internet.

Tomorrow they start feeling smart. TF-IDF, text similarity, and the technique behind resume matchers. The AI days begin.

OPEN THE FLOOR — ASK NOW

- ✓ Did the API call work on your first try, or did you need to debug?
- ✓ Which part was confusing — the params, the JSON response, or the caching?
- ✓ Anything from the exercise that didn't work?

BETWEEN SESSIONS — GET UNSTUCK

- ✓ **Email** · `hello@zlaark.com`
  For API questions, paste your code + the error + the API URL.
- ✓ **Web** · `www.zlaark.com`
  Program updates and resources.
- ✓ **Practice — try the GitHub API. List your own repos.**

WHAT'S NEXT · DAYS 13–14 — AI

- ✓ **Day 13** — TF-IDF + text similarity. How computers compare text.
- ✓ **Day 14** — Build a resume matcher. Real AI-powered tool.
- ✓ **Day 15** — Recap. Then Day 16: group project begins.

Zlaark

See you on Day 13.

Prepared by Zlaark for Hindu College Amritsar
8 August 2026